ALLIANCE UNIVERSITY

Alliance College of Engineering and Design

Student-Centred and Industry-friendly

OBCFC Course Structure

PART B – Micro–Project Report

Micro-project Report after Execution

Course Code: E1CSA25202 **Course Title:** Programming in Python

Name of Course Incharge: Shubha Harini

Library Management System

Student Team Members

1. **V Keerthana**
2. **Koturu Kalpana Reddy**
3. **Gangapalli Rushitha Reddy**
4. **Chavana Yasodha Devi**
5. **Rajavath Jaswanth Naik**
6. **Kaya Mahto**

1.0 Rationale

The Library Management System is a Python-based micro-project developed to automate the core operations of a library — adding books, issuing them to members, accepting returns, and displaying the current availability status of all books. In many institutions, these tasks are still done manually, which leads to errors and the problem of issuing the same book to multiple members at the same time.

This project uses a dictionary (books = {}) to store each book name and its status (Available or Issued). A while loop keeps the program running with a menu-driven interface. The system prevents duplicate issues by checking the current status before issuing. It is built entirely using basic Python — no classes, no external libraries — making it suitable for a beginner-level programming course.

The project bridges the gap between theoretical learning and practical application by applying concepts such as dictionaries, loops, if/elif/else conditions, f-strings, and user input handling in a real-world scenario.

2.0 Competency of the Course

- Develop problem-solving skills using Python programming
- Apply conditional statements and loops to solve real-world problems
- Use data structures (dictionary) for efficient data storage and retrieval
- Design and implement a structured, menu-driven program

3.0 Course Outcomes Addressed

- CO1: Understand Python syntax and basic programming constructs
- CO2: Apply loops, conditional statements, and functions
- CO3: Develop real-world applications using Python
- CO4: Implement data structures for managing data efficiently

4.0 Literature Review

To build this Library Management System, the team studied both library domain knowledge and Python programming concepts from reliable sources.

For domain knowledge, the team examined how books are catalogued, issued, and returned in a typical library setting, identifying the key operations needed: add book, issue book, return book, check availability, and display records.

For technical knowledge, the official Python documentation (docs.python.org) was used to understand dictionaries, loops, and conditional statements. GeeksforGeeks and W3Schools provided practical examples of dictionary operations such as checking membership (in operator), adding key-value pairs, and iterating with .items(). The use of f-strings for formatted output was also studied from these platforms.

Several basic Python mini-projects were reviewed to understand how menu-driven programs are structured using a while True loop with if/elif/else branching. This guided the design of the main program flow and helped in implementing the duplicate issue prevention logic cleanly.

5.0 Actual Methodology Followed

1. Requirement Analysis: Identified required features — add book, issue book with duplicate prevention, return book, display all books with availability status, and exit.
2. System Design: Designed a menu-driven structure using while True loop. Chose a dictionary (books = {}) as the data structure — book name as key, status ('Available'/'Issued') as value.
3. Team Work Distribution:

   V Keerthana – Overall coordination and system integration

   Koturu Kalpana Reddy – Add Book module and empty-name validation

   Gangapalli Rushitha Reddy – Issue Book module with duplicate prevention

   Chavana Yasodha Devi – Return Book module with error handling

   Rajavath Jaswanth Naik – Display Books module with formatted output

   Kaya Mahto – Testing, debugging, and documentation
4. Implementation: Developed each module using basic Python constructs — dictionary, while loop, if/elif/else, f-strings, and input/print statements.
5. Testing and Debugging: Tested each option individually including edge cases — issuing an already-issued book, returning a book not issued, entering empty name, entering invalid menu choice.
6. Final Integration: All modules combined into one single Python file with a clean menu loop.

6.0 Actual Resources Used

S. No.	Name of Resource/material	Specifications	Qty	Remarks
1.	Laptop	Intel i3 or above, 4GB RAM	1 per student	Personal
2.	Python	Version 3.x	1	Open-source (Free)
3.	Code Editor	VS Code / IDLE	1	Free
4.	Internet	For reference and learning	As required	Used

7.0 Outputs of the Micro–project

- Menu-driven Library Management System with 5 options
- Add Book: Adds a new book with status 'Available'; prevents duplicate entries
- Issue Book: Issues a book and changes status to 'Issued'; prevents duplicate issue
- Return Book: Accepts return and resets status to 'Available'; validates before returning
- Display Books: Shows all books with their current availability status in formatted table

Program Code

```
books = {}

while True:

    print("\n" + "=" * 40)
    print("       LIBRARY MANAGEMENT SYSTEM")
    print("=" * 40)
    print("  1. Add Book")
    print("  2. Issue Book")
    print("  3. Return Book")
    print("  4. Display Books")
    print("  5. Exit")
    print("-" * 40)

    choice = input("  Enter your choice (1-5): ")

    if choice == "1":
        print("\n--- Add Book ---")
        name = input("Enter book name: ").strip()
        if name == "":
            print("Book name cannot be empty.")
        elif name in books:
            print("Book already exists in the library.")
        else:
            books[name] = "Available"
            print("Book '" + name + "' added successfully.")

    elif choice == "2":
```

```
    print("\n--- Issue Book ---")
    name = input("Enter book name to issue: ").strip()
    if name not in books:
        print("Book not found in the library.")
    elif books[name] == "Issued":
        print("Book is already issued. Cannot issue again.")
    else:
        books[name] = "Issued"
        print("Book '" + name + "' issued successfully.")

elif choice == "3":
    print("\n--- Return Book ---")
    name = input("Enter book name to return: ").strip()
    if name not in books:
        print("Book not found in the library.")
    elif books[name] == "Available":
        print("Book was not issued. Cannot return.")
    else:
        books[name] = "Available"
        print("Book '" + name + "' returned successfully.")

elif choice == "4":
    print("\n--- Books List ---")
    if not books:
        print("No books available in the library.")
    else:
        print(f"{'Book Name':<30} {'Status'}")
        print("-" * 40)
        for book, status in books.items():
            print(f"{book:<30} {status}")
        print("-" * 40)
        print("Total Books:", len(books))

elif choice == "5":
    print("\nThank you for using the Library Management System!")
    print("=" * 40)
    break

else:
    print("Invalid choice. Please enter a number between 1 and 5.")
```

Code Explanation

1. Data Structure

books = {} — A dictionary where each key is the book name (string) and the value is the status string: either 'Available' or 'Issued'.

2. while True Loop

The program runs continuously in a while True loop, displaying the menu and accepting user input each iteration. The loop only exits when the user enters choice 5 and the break statement executes.

3. Add Book (Choice 1)

Takes the book name as input. First checks if the name is empty (empty string validation). Then checks if the book already exists in the dictionary (duplicate prevention). If neither, adds books[name] = 'Available'.

4. Issue Book (Choice 2)

Takes book name as input. Checks if book is in the dictionary — if not, prints 'Book not found'. Then checks if books[name] == 'Issued' — if yes, prints 'Already issued. Cannot issue again.' (duplicate issue prevention). Otherwise, sets books[name] = 'Issued'.

5. Return Book (Choice 3)

Takes book name as input. Checks if book exists in dictionary. Then checks if books[name] == 'Available' — if yes, book was never issued so prints error. Otherwise, sets books[name] = 'Available'.

6. Display Books (Choice 4)

Checks if the dictionary is empty first. If not empty, prints a formatted table using f-strings with book name (left-aligned in 30 characters) and status. Also prints the total count of books using len(books).

7. Exit (Choice 5)

Prints a goodbye message and executes break to exit the while True loop, ending the program.

Sample Inputs and Outputs

1. Add Book

Input: choice = 1, name = Programming in Python
Output: Book 'Programming in Python' added successfully.

Input: choice = 1, name = Programming in Python (again)
Output: Book already exists in the library.

2. Issue Book

Input: choice = 2, name = Programming in Python
Output: Book ' Programming in Python ' issued successfully.

Input: choice = 2, name = Programming in Python (again - duplicate attempt)
Output: Book is already issued. Cannot issue again.

3. Return Book

Input: choice = 3, name = Programming in Python
Output: Book ' Programming in Python ' returned successfully.

Input: choice = 3, name = XYZ Book (not in library)
Output: Book not found in the library.

4. Display Books

Input: choice = 4

Output:

```
--- Books List ---
Book Name                    Status
---------------------------------------------------
Programming in Python        Available
OOPS                         Issued
---------------------------------------------------
Total Books: 2
```

Findings

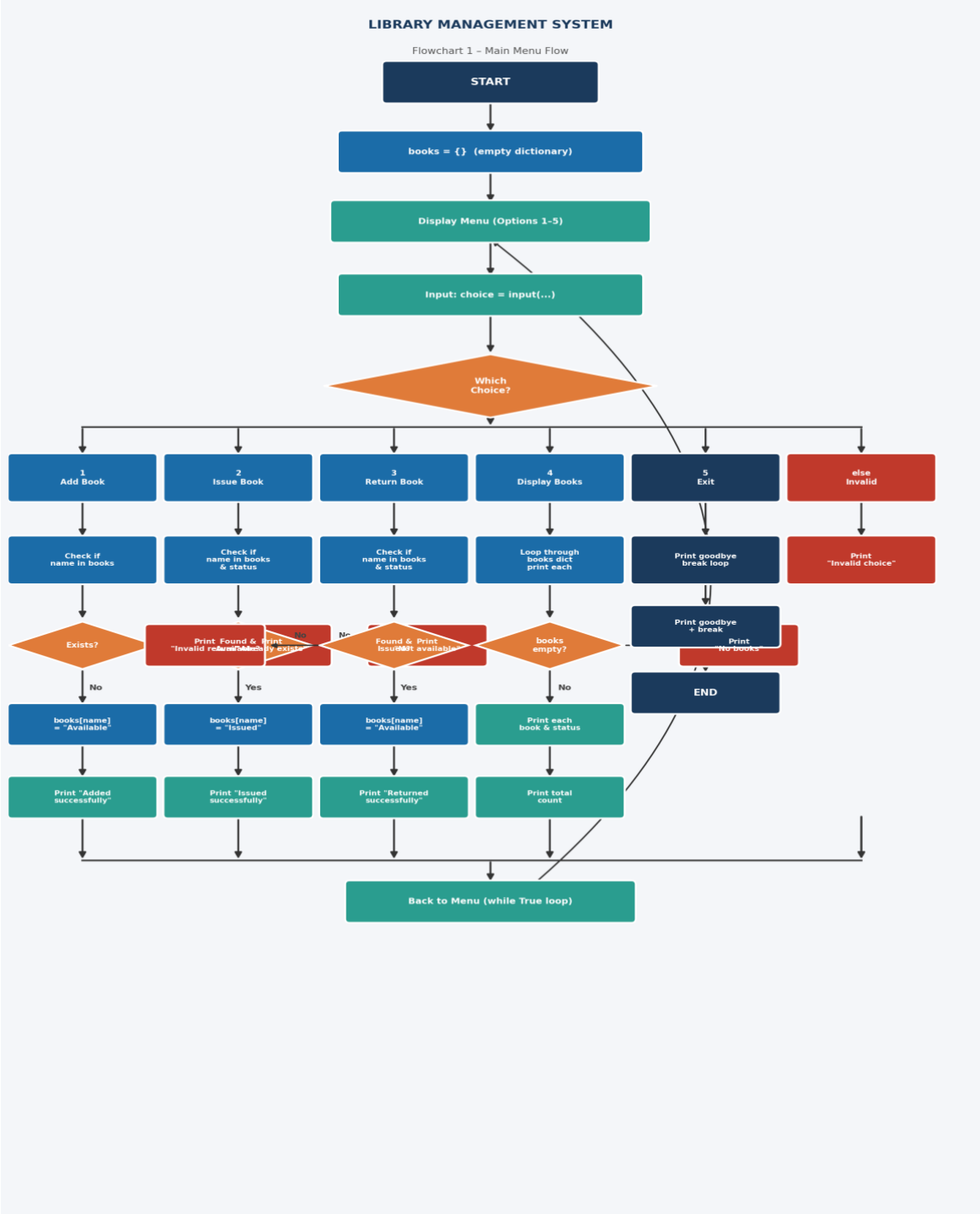
- Dictionary is an efficient data structure for storing and retrieving book records
- while True loop with if/elif/else provides a clean menu-driven interface
- Duplicate issue prevention works by checking books[name] == 'Issued' before issuing
- f-strings make the display output neatly formatted and readable
- Python is very effective for building small-scale management systems with basic constructs

Limitations

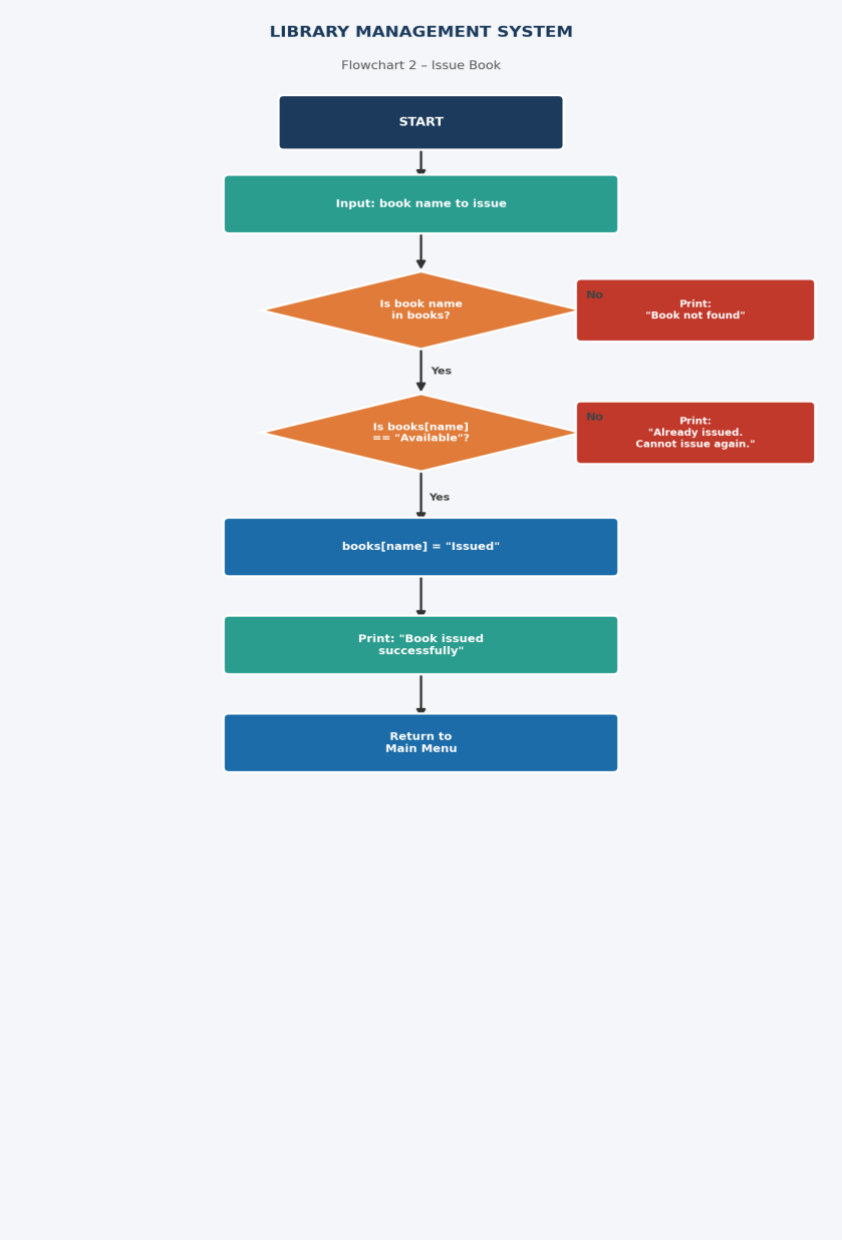
- No persistent storage — all book data is lost when the program exits
- No graphical user interface (GUI); runs only in command line
- Book name is the key, so two books with the same name cannot be stored separately
- No member name tracking — does not record who borrowed a book

Flowcharts

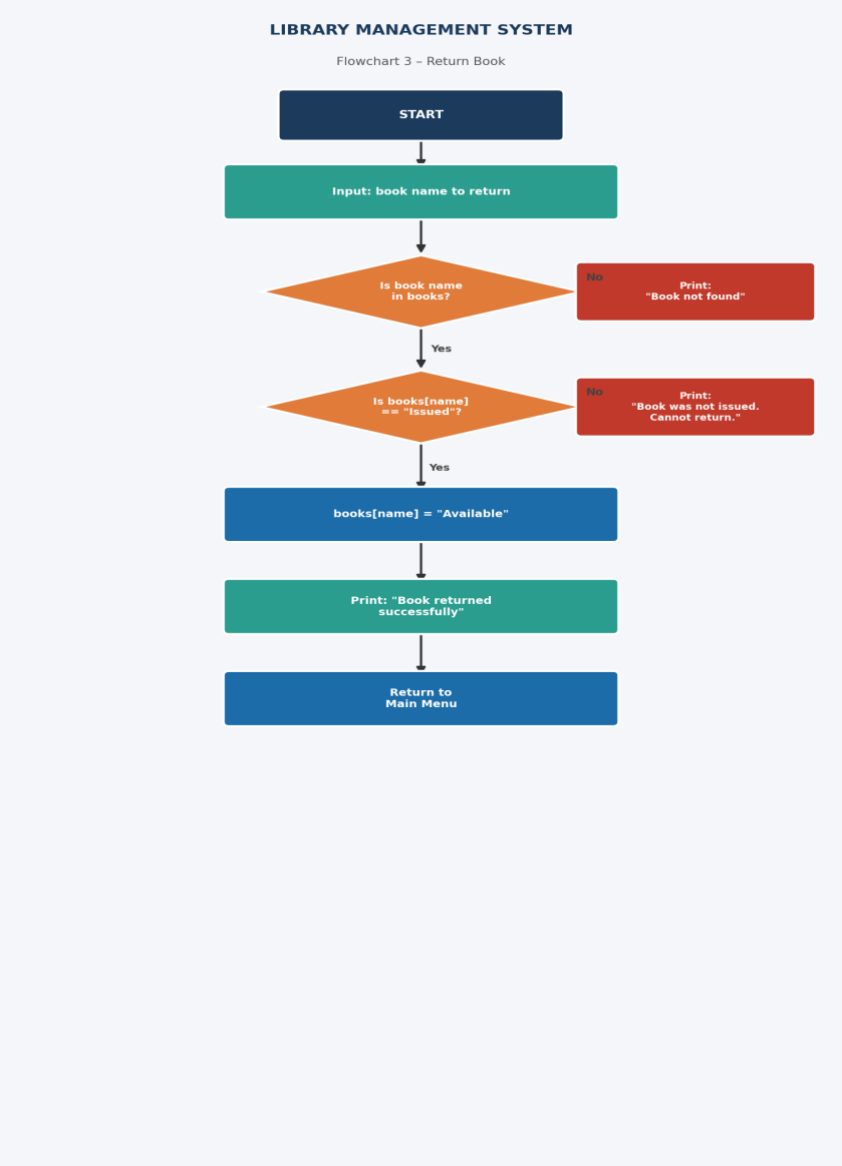
Flowchart 1 – Main System Flow



Flowchart 2 – Issue Book



Flowchart 3 – Return Book



8.0 Benefits of this Micro–project

This project improves practical programming skills by applying basic Python concepts — dictionaries, loops, conditionals, and f-strings — to a real-world library problem. It develops logical thinking and understanding of how data is stored and managed using key-value pairs. The team gained hands-on experience in modular thinking, input validation, and structured program design. It also provides practice in collaborative work by dividing the development into clear individual responsibilities for all six team members.

9.0 Area of Future Improvement

The system can be enhanced by using a file (file handling) to store book records permanently so data is not lost when the program exits. Member name tracking can be added so the system records who has borrowed each book. A due date feature and fine calculation for overdue books can be included. In future advanced courses, the system can be upgraded with a GUI using Tkinter and a database using SQLite for larger-scale library management.

10.0 References

[1] Python Software Foundation, Python Documentation. (Online). Available: https://docs.python.org

[2] GeeksforGeeks, Python Programming Language. (Online). Available: https://www.geeksforgeeks.org

[3] W3Schools, Python Tutorial. (Online). Available: https://www.w3schools.com/python

[4] Python Dictionary Methods – GeeksforGeeks. (Online). Available: https://www.geeksforgeeks.org/python-dictionary

****